

Are Identifier Types Identified?

Linear probing of CodeLLMs for class, variable and function features

Lois Lagerweij, Filip Szutowicz, Lisa Douwes

Institute for Logic, Language and Computation (ILLC)

1 Introduction

The field of mechanistic interpretability, which aims to understand how machine learning models *actually* work, has been growing in popularity [Somvanshi et al., 2025]. However, due to the recency of the field, many applications remain unexplored. In the context of codeLLMs (Large Language Models trained or finetuned using code-data) prior research has for example found that semantic attributes like code correctness and syntactic attributes like the number of open parentheses are linearly represented within the model [Tahimic and Cheng, 2025a, Miller et al., 2025].

Prior research has probed for ‘token distance in an abstract syntax tree (AST)’ and found that cluster formation was of poor quality [Anand et al., 2024]. In particular, the study failed to find a representation of AST token distance for identifiers (that is, names of functions, classes and variables). This study concluded that identifier types, which are encompassed by an AST, are thus not encoded in the investigated models. However, this paper did not attempt probing on identifier types *directly*. We hypothesize that identifier types such as variables, functions and classes, might be directly encoded. This hypothesis is supported by a paper that has found that part-of-speech (POS) tags, which are also explicit classes describing syntactic structure, are linearly represented in some LLMs [Köhn, 2015].

While other research has focused on encoding the syntactic relations amongst tokens, the possibility of identifier types being directly encoded has not been studied [Anand et al., 2024]. This research report aims to address this knowledge gap. We want to find out whether identifier-type features (function, class, variable) are encoded linearly inside (some layers) of the CodeLlama-7-hf model. We use linear probing and steering experiments to address this question. We find evidence suggesting that identifier types are linearly encoded in LLMs, and demonstrate a causal effect of the probed class direction in the steering experiment.

2 Methods

In order to inspect the encoding of identifier types, a linear probe was trained using hidden activations cached from running a fill-in-the-middle (FIM) task on CodeLlama-7-hf with frozen weights. CodeLlama-7b is a 7b parameter Code LM with a 32-layer decoder-only transformer architecture. Specifically, the base model CodeLlama-7b-hf is used. In this section, we first introduce the training

data used, after which the details of the linear probe and causal experiments are explained.

2.1 Training data

CodeLlama-7-hf is trained on Python code with the FIM task using special tokens, which we employed in making training data for training the probe. In the FIM setup, the prompt is structured as in the following example:

```
<PRE>def <SUF>(j, c):
    return j + c
w = g(o, k)<MID>
```

The model is trained so that the next generated tokens correspond to the missing middle segment (what has to be filled in at the location of `<SUF>`). In our setting, the missing middle always consists of an identifier that is either a function name, class name, or variable name. Consequently, the token that the model must predict always corresponds to a specific identifier type.

The training data contains one instantiation of an identifier (defining a function, declaring a variable, or defining a class), followed by a use of that identifier, such as calling the function, instantiating the class, or referencing the variable. This identifier is then masked in either its instantiation or usage. In order to eliminate ambiguity in the syntax regarding the use of functions and classes, a random method or attribute of the class is called. The contents of the classes and functions, as well as the number of inputs, methods and attributes of the function and class was randomised. Lastly, the datatype of the variables was randomized (between int, float, list, string, dictionary or set). To ensure the probe does not classify based on the semantic information of the identifiers (e.g. class being called MyClass), all identifiers are one random unique letter.

Probes are trained on two different datasets. First, a probe is trained on non-contrastive data with the definition and usage of only one identifier type in each example. Secondly, another probe is trained on contrastive examples containing the definition and usage of all three different identifier types, in which the order in which the identifier types get defined and used is randomised. Training data is generated using the LLM Gemini. This is done by prompting the model with a template and explanation of how the data should be randomised and diversified. See Appendix for the template and the GitHub Repository for the full datasets ¹.

2.2 Probing and causality experiments

In order to investigate whether identifier types are encoded in the model, we train a linear probe [Alain and Bengio, 2018]. A linear probe is a linear classifier that takes a hidden activation vector from a particular layer as input ($\mathbf{x}$) and produces logits ($\mathbf{y}$) over a set of classes.

In this work, the probe is implemented as multinomial logistic regression:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b},$$

where W and $\mathbf{b}$ are learned using cross-entropy loss. In order to retrieve a single

¹ <https://github.com/lois-mla/MechInt-project-probing-identifier-types>

activation vector, a specific token position and a specific layer are fixed for all training data of the linear probe. The classes then correspond to the encoded features to be investigated. The rows of the learned weight matrix W can be interpreted as linear directions in activation space associated with the different features.

In this project, the activations of the last token of the prompt get cached, since these are used to predict the next token, and thus contain the relevant information for the token to be predicted, possibly including its identifier type [Tahimic and Cheng, 2025b]. The activations are cached after the MLP-block, before the result of the block gets added to the residual stream. This enables investigation of which MLP blocks in which layers use identifier type information, with a less cluttered activation-space compared to caching the residual stream directly.

There are several methods to find candidate layers for probing, e.g. logit-lens experiments [Somvanshi et al., 2025]. However, due to the relatively small size of the model used, which has 32 layers in total, probing was performed in all layers. The accuracy of the linear probe on test-data is then an indication for how much information about the feature the MLP block of this layer linearly encodes.

Causality experiments such as activation steering can test whether the directions found with probing causally encode this feature: whether they are in practice used by the model [Bricken et al., 2023]. To do so, the activations are manipulated using the found feature direction(s). In this project, contrastive steering was used, which steers from one feature direction (A) to another (B), using the steering vector $\mathbf{s} = \mathbf{w}_B - \mathbf{w}_A$, where $\mathbf{w}_B$ is the row of the linear probe matrix W corresponding to the B class. At the probing location, the hidden activations are changed as follows: $\mathbf{a}_{steered} = \mathbf{a} + \alpha \hat{\mathbf{s}}$, where α is a hyperparameter and $\hat{\mathbf{s}}$ is the unit-length steering vector. We present the results for $\alpha = 100$, which gave the best results. We also conducted experiments for α values 10, 30, 50 and 200. For the steering experiments, a new dataset was created with analogous examples to the training dataset, using contrastive cases for both probes, in order to be able to analyse the effect of steering. For each example, two steering experiments are conducted. That is, we steer in the direction of the two other identifier types.

For each steering direction from identifier type A to identifier type B , let ID_A and ID_B denote the names of identifier type A and B respectively for some example. A quantitative analysis is conducted to investigate how the probabilities of ID_A and ID_B change when steering is applied to some layer compared to the baseline, where no intervention is done. For each of the six steering directions, the effectiveness of steering at a specific layer was calculated as follows: $avg((p_s(ID_B) - p_s(ID_A)) - (p_b(ID_B) - p_b(ID_A)))$, following [Marks and Tegmark, 2023]. In this formula, $p_s(ID_A)$ is the softmax of the logit value of ID_A for the steered run, and p_b is the baseline run. The average is taken over all the data where an identifier-name of type A should be filled in. The formula can be interpreted as the sum of two measures, namely $avg(p_s(ID_B) - p_b(ID_B))$ (how much more likely ID_B becomes) and $avg(p_b(ID_A) - p_s(ID_A))$ (how much less likely ID_A becomes). A positive score thus shows that the probability of ID_B

has increased and/or the probability of ID_A has decreased as a result of the steering intervention. In order to get insight into the contributions of both these components, their values per layer will also be plotted.

3 Results

Figure 1 shows the training and testing accuracies of the linear probes across the layers of the model. We conducted steering experiments for six steering directions and five values of α (10, 30, 50, 100 and 200). We found that generally an α value of 100 produced the most prominent results. Furthermore, the steering experiment on this data was effective for only two steering directions, namely from *variable to class* and from *function to class*. The probability of the contrastive identifier type in those examples rose as much as 0.25 in layer 7 for steering from variable towards class (Figure 2a). For data without contrastive examples, hardly any experiment showed significant results, with highest increases in the probability of the contrastive identifier type of the magnitude of 0.05 for $\alpha = 100$. Figure 2 and Figure 3 show the steering results for the *variable to class* and *function to class* steering directions for the contrastive and non-contrastive probe for $\alpha = 100$.

Results for all steering directions and values of α are available on the GitHub Repository.

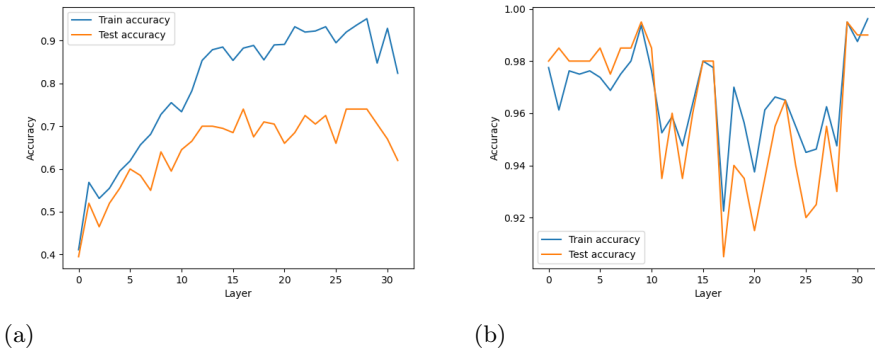


Fig. 1: Classification accuracy of the linear probe trained on the contrastive dataset (a) and the non-contrastive dataset (b).

4 Analysis and Discussion

4.1 Probing

Figure 1 shows the probe accuracies, which is an indication for how much information about the feature the MLP block of this layer linearly encodes. In Figure 1, it can firstly be seen that the train and test accuracy for the probe trained on the non-contrastive dataset are substantially higher. This could be explained by the fact that the examples are shorter and the probed activations therefore encode less information.

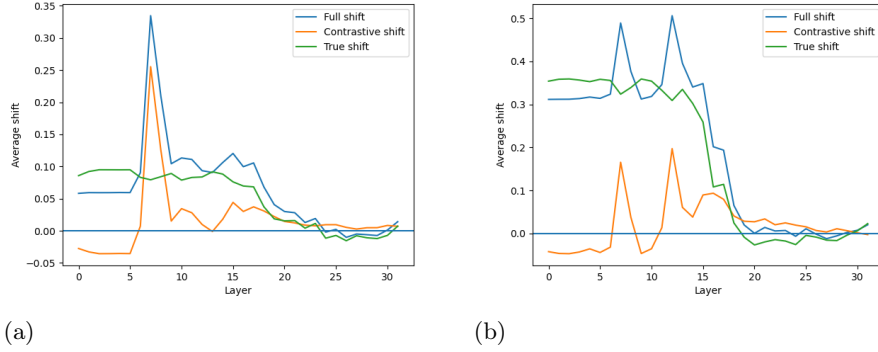


Fig. 2: Average steering effect for steering intervention in each layer respectively. Values represent the difference between probability of given token with and without—steering. In 2a steering from variable towards class, in 2b from function to class. **Contrastive shift** indicates the change in the probability of the token we steer towards. **True shift** shows the inverse of the change in the probability of the correct token, so the higher it is the more we managed to steer away from that token. **Full shift** is the sum of the previous two values.

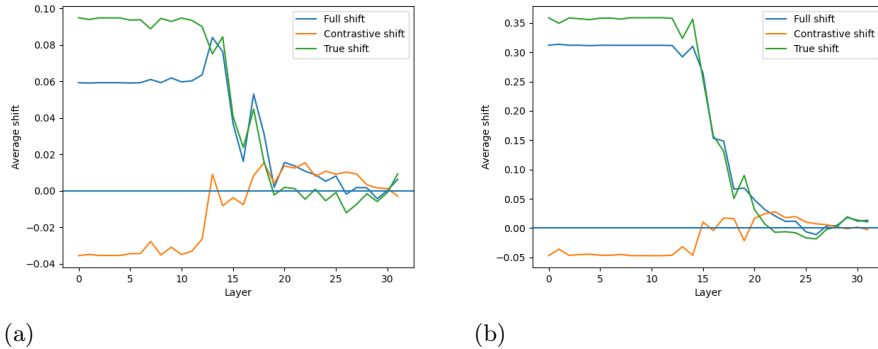


Fig. 3: Same as Figure 2, but for the non-contrastive dataset. In 3a steering from variable towards class, and in 3b from function to class.

Secondly, it is notable that for the probe trained on the contrastive dataset, the accuracy generally increases over the layers. At first sight, this is surprising, since the activations were cached right after the MLP, before this result is added to the residual stream. However, the input to each MLP already contains information accumulated from earlier layers throughout the residual stream. These representations are then transformed or refined. This rising trend could thus suggest that over the layers, the information becomes more linearly separable. In this case, the stark increase in the earlier layers of the contrastive probe suggests that those layers contribute the most to extracting the information about the different identifier types. Note that the conducted probing experiments

presuppose that the identifier-type information for *all* identifier-types are encoded in the same layer(s). However, it could be the case that the variable, function and class features are encoded in different layers, in which case the probe will generally be unable to classify the identifier-types based on the activations of one layer or will classify them based on other information encoded in the probed activations.

4.2 Causal experiments

Figure 2a and 2b show the steering results for the probe trained on the contrastive dataset for the *variable to class* and *function to class* steering direction respectively. It can be seen that for the *variable to class* steering direction, steering in layer 7 is effective. For the *function to class* steering direction, steering was mainly effective in layer 7 and 12. Namely, in these layers, both the contrastive shift and true shift are positive. These results show that using the found class, variable and function direction, we can successfully steer towards *class*, but we cannot effectively steer in any other directions.

Figure 3a and 3b show the same steering directions, but for the probe trained on the non-contrastive dataset. It can be seen that for this probe, the same steering experiments produce no significant results: the contrastive shift is negligible, and the true shift can also be caused by arbitrary changes to the activations, and do therefore not point towards any influence of identifier-type information. This suggests that none of the found feature-directions of the non-contrastive probe are causal in nature. However, for the probe trained on the contrastive dataset, causal feature directions *were* found. The combination of a very high test-accuracy across all layers, and the lack of significant steering results while a differently trained probe did find results, suggests that the non-contrastive dataset is inadequate. The reason behind that might be the difference between the non-contrastive data and the steering data. The identifiers occur without a larger context in the dataset without contrastive data, while they are surrounded by ‘noise’ in the steering data. Thus the representation (feature direction) of each identifier, possibly being based on context, will diverge between the contrastive and non-contrastive data, with the one found in contrastive data more similar to the one necessary for steering.

There are several possible explanations for the fact that no other steering directions were effective. It could be the case that the variable and function identifier features are not, or less strongly, encoded and used in the model. However, it could also be the case that our probing and steering experiments simply did not manage to find these causal features.

4.3 Conclusions and future research

In conclusion, we find evidence suggesting that identifier types are linearly encoded in LLMs, and demonstrate a causal effect of the probed class direction in the steering experiment. For the other identifier types, a causal effect could not be found using our specific implementation. These findings opens up fruitful directions for future research. Firstly, future research could focus on experimenting with more different datasets for training the probe. Contrary to training

procedures for ‘regular’ ML models, for linear probing, more diverse training data is not always better. However, the generalizability of the probing results is shown to improve for more diverse datasets [Marks and Tegmark, 2023]. Nonetheless, datasets for linear probing do generally follow a rigid template, where the prompts are all precisely the same except for certain words that get varied (see [Tigges et al., 2023], [Marks and Tegmark, 2023]). This high similarity in training data increases the chances that the feature directions are found by the probe, since ideally the only meaningful difference in activations between the different prompts is whether the investigated features are present. However, applying this methodology to the case of identifier types is more complicated due to its *syntactic nature*; instead of varying some word (e.g. substitute some positive word [Tigges et al., 2023]), the contents of the function, class and variable need to be varied. This means that the difference in information across examples of the same identifier type is greater, decreasing the chances of finding the desired features.

As well as this, future research could focus on trying different types of probes to find the identifier-type feature directions. Firstly, due to the high dimensionality of the activation-space, using a one-versus-all linear classifier may produce better results than using multinomial logistic regression [Köhn, 2015]. This would also help the previously stated problem about all identifier types having to be encoded in the same layer. Secondly, it could be investigated whether the identifier, variable and class features are represented non-linearly, by employing a probe that also looks for non-linear features, such as DirectProbe [Zhou and Srikumar, 2021].

Thirdly, future research could focus on improving the steering experiments, since the lack of results for other steering directions may be due to the nature of our specific steering experiment. Firstly, we use contrastive steering, which relies on two feature-directions at once. This means that in case only one feature-direction (information of only one identifier type) is prominent in some layer, this technique is less likely to give effective results. Secondly, the conducted steering experiment only steers in one layer at once. This means that in case the feature is encoded in several layers, steering in only one layer may not prove to be very effective. Current research on cross layer superposition might help shed light on this issue [Lindsey et al., 2024].

References

- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018. URL <https://arxiv.org/abs/1610.01644>, 2018.
- Abhinav Anand, Shweta Verma, Krishna Narasimhan, and Mira Mezini. A critical study of what code-llms (do not) learn. *arXiv.org*, 2024. URL <https://arxiv.org/abs/2406.11930>.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christo-

- pher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Arne Köhn. What’s in an embedding? analyzing word embeddings through multilingual evaluation. 2015.
- Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse crosscoders for cross-layer features and model diffing, Oct 2024. URL <https://transformer-circuits.pub/2024/crosscoders/index.html>.
- Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*, 2023.
- Samuel Miller, Daking Rai, and Ziyu Yao. Mechanistic understanding of language models in syntactic code completion. *arXiv.org*, 2025. URL <https://arxiv.org/abs/2502.18499>.
- Shriyank Somvanshi, Md Monzurul Islam, Amir Rafe, Anannya Ghosh Tusti, Arka Chakraborty, Anika Baitullah, Tausif Islam Chowdhury, Nawaf Alnawmasi, Anandi Dutta, and Subasish Das. Bridging the black box: A survey on mechanistic interpretability in ai. *ACM Computing Surveys*, 2025.
- Kriz Tahimic and Charibeth Cheng. Mechanistic interpretability of code correctness in llms via sparse autoencoders. *arXiv preprint arXiv:2510.02917*, 2025a.
- Kriz Tahimic and Charibeth Cheng. Mechanistic interpretability of code correctness in llms via sparse autoencoders. *arXiv.org*, 2025b. URL <https://arxiv.org/abs/2510.02917>.
- Curt Tigges, Oskar John Hollinsworth, Atticus Geiger, and Neel Nanda. Linear representations of sentiment in large language models. *arXiv preprint arXiv:2310.15154*, 2023.
- Yichu Zhou and Vivek Srikumar. Directprobe: Studying representations without classifiers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5070–5083, 2021.

5 Appendix

Template for the training data. There are 6 options for the fill-in-the-middle token (FIM) placement, as indicated by the auxiliary comments that are not present in the actual data. The line with ‘ID’ at the end indicates the identifier type of the masked token, with 0 for variable, 1 for function and 2 for class.

```
def g(j, c):      # mask 1: function name (g)
    return j + c
class FIM:       # mask 2: class name (l)
    a = 5
k = 3            # mask 3: variable name (k)
h = l()         # mask 4: class name (l)
```



```
o = h.a          # attribute access
w = g(o, k)      # mask 5: function name (g)
d = k + 3        # mask 6: variable name (k)
>>> 1
ID:0
####
```